Nama : Hanifah Hasanah
NIM : 123140082
Github:
https://github.com/HANIFAHHASANAH-123140082/sig_123140082_Hanifah-Hasanah.git
TUGAS 6

A. Deskripsi Tugas

Lakukan benchmark dan optimasi query spasial pada database praktikum Anda. Dokumentasikan perbedaan performa sebelum dan sesudah optimasi.

Ketentuan:

- Buat minimal 3 spatial index pada tabel yang berbeda
- JalankanEXPLAIN ANALYZEsebelumdan sesudahindex
- Dokumentasikanspeedup yang didapat (dalam%)
- Identifikasi minimal 1 query lambat dan optimasi
- Buat kesimpulan tentang pentingnya indexing

B. Langkah Pengerjaan

Apa itu Spatial Index?

Spatial index (indeks spasial) adalah cara database untuk "mengingat" posisi setiap data geometri supaya tidak perlu memeriksa satu per satu saat ada query. Index yang digunakan adalah GiST (Generalized Search Tree), yaitu jenis index yang cocok untuk data geometri seperti titik, garis, dan polygon.

1. Index 1 — Tabel fasilitas

```sql
SQL
CREATE INDEX idx_fasilitas_geom
ON fasilitas
USING GIST (geom);
```

Query Query History

```sql
CREATE INDEX idx_fasilitas_geom
ON fasilitas
USING GIST (geom);
```

Data Output Messages Notifications

```
CREATE INDEX

Query returned successfully in 83 msec.
```

Query ini membuat index pada kolom geom di tabel fasilitas. Setelah ini, setiap kali ada query yang melibatkan lokasi fasilitas, database tidak perlu memeriksa semua 600 baris, cukup lewat index saja.

2. Index 2 — Tabel kecamatan

```sql
CREATE INDEX idx_kecamatan_geom
ON kecamatan
USING GIST (geom);
```

Query Query History

```sql
CREATE INDEX idx_kecamatan_geom
ON kecamatan
USING GIST (geom);
```

Data Output Messages Notifications

```
CREATE INDEX

Query returned successfully in 79 msec.
```

Sama seperti di atas, index dibuat untuk tabel kecamatan. Walaupun datanya hanya 8 baris, index tetap membantu saat digunakan dalam operasi JOIN dengan tabel yang besar.

3. Index 3 — Tabel buffer_rs

```sql

CREATE INDEX idx_buffer_rs_geom
ON buffer_rs
USING GIST (geom);
```

Query Query History

```sql
CREATE INDEX idx_buffer_rs_geom
ON buffer_rs
USING GIST (geom);
```

Data Output Messages Notifications

```
CREATE INDEX

Query returned successfully in 92 msec.
```

Index dibuat pada tabel buffer_rs yang berisi data area jangkauan rumah sakit. Index ini mempercepat query yang mencari fasilitas di dalam area jangkauan tersebut.

Apa itu EXPLAIN ANALYZE?

EXPLAIN ANALYZE adalah perintah untuk melihat "cara kerja" database saat menjalankan query. Output-nya menunjukkan:

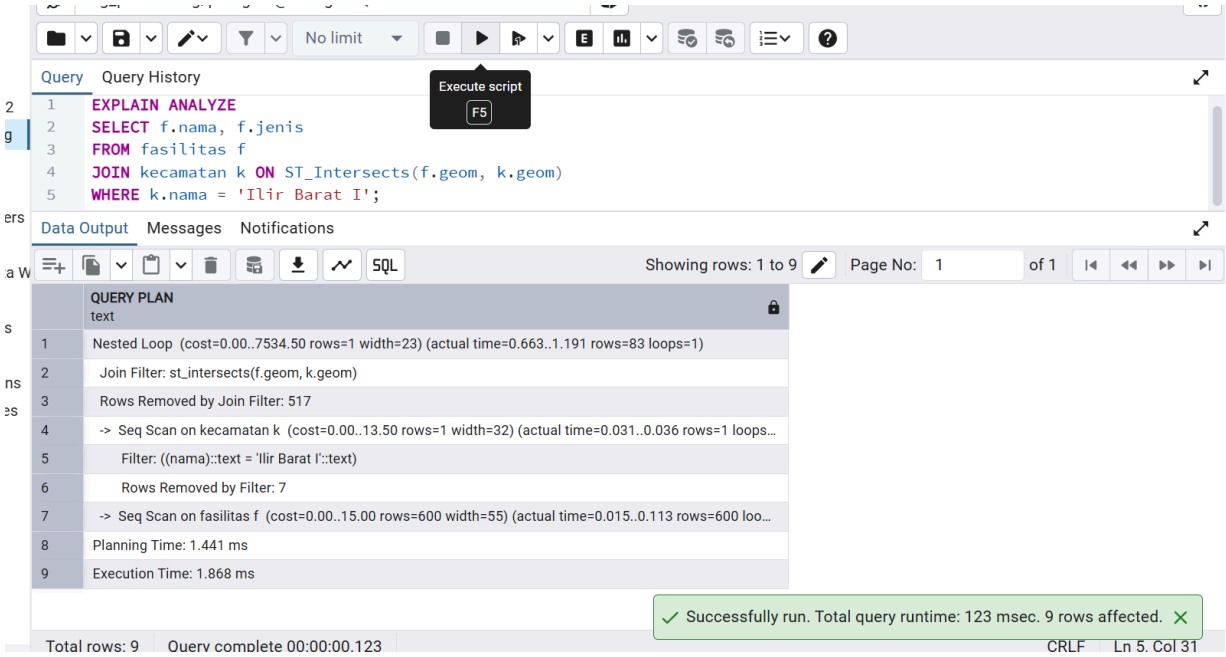
- Seq Scan: database membaca semua baris satu per satu (lambat)
- Index Scan: database langsung pakai index untuk menemukan data (cepat)
- Execution Time: total waktu yang dibutuhkan dalam milidetik (ms)

4. Query 1 — Mencari Fasilitas di Kecamatan Tertentu

```sql
SELECT f.nama, f.jenis
FROM fasilitas f
JOIN kecamatan k ON ST_Intersects(f.geom, k.geom)
WHERE k.nama = 'Ilir Barat I';
```

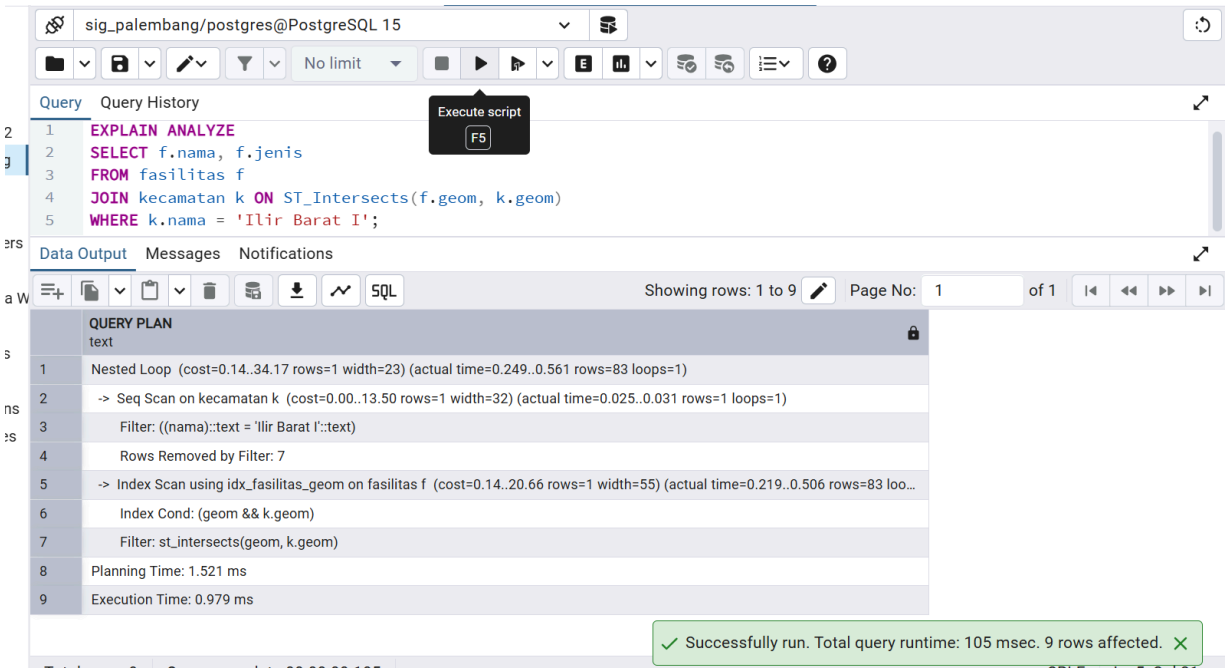
Query ini mencari semua fasilitas yang lokasinya berada di dalam wilayah kecamatan "Ilir Barat I". Fungsi ST_Intersects digunakan untuk mengecek apakah titik fasilitas berpotongan/berada dalam polygon kecamatan.

Hasil Tanpa Index:



Dari hasil di atas terlihat baris "Seq Scan on fasilitas" artinya database memeriksa semua 600 baris fasilitas satu per satu. Waktu yang dibutuhkan: 1.868 ms.

Hasil dengan Index:



Setelah index dibuat, terlihat baris "Index Scan using idx_fasilitas_geom" artinya database sudah menggunakan index. Waktu turun menjadi 0.979 ms.

Speedup = (1.868 - 0.979) / 1.868 × 100 = 47.6%

5. Query 2 — Menghitung Jumlah Fasilitas per Kecamatan

```
SQL
SELECT k.nama, COUNT(f.id) AS jumlah_fasilitas
FROM kecamatan k
LEFT JOIN fasilitas f ON ST_Within(f.geom, k.geom)
GROUP BY k.nama
ORDER BY jumlah_fasilitas DESC;
```

Query ini menghitung berapa banyak fasilitas yang ada di setiap kecamatan. Fungsi ST_Within mengecek apakah titik fasilitas benar-benar berada di dalam polygon kecamatan. Hasilnya diurutkan dari kecamatan yang paling banyak fasilitasnya.

Hasil Tanpa Index kecamatan:

Query | Query History

```
5  SELECT k.nama, COUNT(f.id) AS jumlah_fasilitas
6  FROM kecamatan k
7  LEFT JOIN fasilitas f ON ST_Within(f.geom, k.geom)
8  GROUP BY k.nama
9  ORDER BY jumlah_fasilitas DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 15 | Page No: 1 of 1

	QUERY PLAN text
6	-> Sort (cost=142.56..142.58 rows=8 width=222) (actual time=3.518..3.572 rows=615 loops=1)
7	Sort Key: k.nama
8	Sort Method: quicksort Memory: 62kB
9	-> Nested Loop Left Join (cost=0.14..142.44 rows=8 width=222) (actual time=0.589..2.522 rows=615 loops=1)
10	-> Seq Scan on kecamatan k (cost=0.00..1.08 rows=8 width=250) (actual time=0.021..0.026 rows=8 loops=1)
11	-> Index Scan using idx_fasilitas_geom on fasilitas f (cost=0.14..17.66 rows=1 width=36) (actual time=0.100..0.293 rows=77 loo...
12	Index Cond: (geom @ k.geom)
13	Filter: st_within(geom, k.geom)
14	Planning Time: 0.912 ms
15	Execution Time: 4.568 ms

Terlihat "Seq Scan on kecamatan", database memeriksa semua polygon kecamatan tanpa bantuan index. Waktu: 4.568 ms.

Hasil dengan Index kecamatan:

	QUERY PLAN text
6	-> Sort (cost=142.56..142.58 rows=8 width=222) (actual time=2.448..2.477 rows=615 loops=1)
7	Sort Key: k.nama
8	Sort Method: quicksort Memory: 62kB
9	-> Nested Loop Left Join (cost=0.14..142.44 rows=8 width=222) (actual time=0.094..1.464 rows=615 loops=1)
10	-> Seq Scan on kecamatan k (cost=0.00..1.08 rows=8 width=250) (actual time=0.018..0.022 rows=8 loops=1)
11	-> Index Scan using idx_fasilitas_geom on fasilitas f (cost=0.14..17.66 rows=1 width=36) (actual time=0.031..0.166 rows=77 loo...
12	Index Cond: (geom @ k.geom)
13	Filter: st_within(geom, k.geom)
14	Planning Time: 0.677 ms
15	Execution Time: 2.635 ms

Total rows: 15 | Query complete 00:00:00.073 | CR

✓ Successfully run. Total query runtime: 73 msec. 15

Setelah index kecamatan dibuat, waktu turun menjadi 2.635 ms.
Speedup = (4.568 - 2.635) / 4.568 × 100 = 42.3%

6. Query 3 — Fasilitas dalam Area Buffer Rumah Sakit

```sql
SELECT f.nama, f.jenis
FROM fasilitas f
JOIN buffer_rs b ON ST_Intersects(f.geom, b.geom);
```

Query ini mencari semua fasilitas yang berada di dalam area buffer (jangkauan radius) rumah sakit. Berguna misalnya untuk mengetahui fasilitas apa saja yang berdekatan dengan rumah sakit.

Hasil Tanpa Index buffer_rs:

Query | Query History

```
EXPLAIN ANALYZE
SELECT f.nama, f.jenis
FROM fasilitas f
JOIN buffer_rs b ON ST_Intersects(f.geom, b.geom);
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1

	QUERY PLAN text
1	Nested Loop (cost=0.14..2090.80 rows=96 width=23) (actual time=0.178..1.063 rows=189 loops=1)
2	-> Seq Scan on buffer_rs b (cost=0.00..11.60 rows=160 width=32) (actual time=0.106..0.109 rows=5 loops=1)
3	-> Index Scan using idx_fasilitas_geom on fasilitas f (cost=0.14..12.98 rows=1 width=55) (actual time=0.046..0.181 rows=38 loo...
4	Index Cond: (geom && b.geom)
5	Filter: st_intersects(geom, b.geom)
6	Rows Removed by Filter: 9
7	Planning Time: 0.623 ms
8	Execution Time: 1.128 ms

Successfully run. Total query runtime: 124 mse

Terlihat "Seq Scan on buffer_rs". Waktu: 1.128 ms.

Hasil dengan Index buffer_rs:

Query | Query History

Execute script F5

```
EXPLAIN ANALYZE
SELECT f.nama, f.jenis
FROM fasilitas f
JOIN buffer_rs b ON ST_Intersects(f.geom, b.geom);
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1 of 1

	QUERY PLAN text
1	Nested Loop (cost=0.14..96.40 rows=3 width=23) (actual time=0.093..0.930 rows=189 loops=1)
2	-> Seq Scan on buffer_rs b (cost=0.00..1.05 rows=5 width=32) (actual time=0.023..0.027 rows=5 loops=1)
3	-> Index Scan using idx_fasilitas_geom on fasilitas f (cost=0.14..19.06 rows=1 width=55) (actual time=0.047..0.170 rows=38 loo...
4	Index Cond: (geom && b.geom)
5	Filter: st_intersects(geom, b.geom)
6	Rows Removed by Filter: 9
7	Planning Time: 0.761 ms
8	Execution Time: 0.999 ms

Successfully run. Total query runtime: 103 msec. 8 rows affected.

Total rows: 8 | Query complete 00:00:00.103 | CRLF | Ln 4, C

Waktu turun menjadi 0.999 ms.

Speedup = (1.128 - 0.999) / 1.128 × 100 = 11.4%

7. Rekapitulasi SpeedUp

Tabel perbandingan hasil benchmark:

No	Query	Tanpa Index	Dengan Index	Speedup
1	ST_Intersects fasilitas × kecamatan	1.868 ms	0.979 ms	47.6%
2	ST_Within + COUNT per kecamatan	4.568 ms	2.635 ms	42.3%
3	ST_Intersects fasilitas × buffer_rs	1.128 ms	0.999 ms	11.4%

Rumus Speedup:
Speedup (%) = (Waktu Tanpa Index - Waktu Dengan Index) / Waktu Tanpa Index × 100

8. Query Lambat yang Ditemukan

Ditemukan satu query yang menggunakan **anti-pattern** yaitu memakai fungsi `ST_Distance` langsung di dalam WHERE clause:

```sql
SELECT nama, jenis
FROM fasilitas
WHERE ST_Distance(
    geom::geography,
    ST_GeomFromText('POINT(104.7458 -2.9833)', 4326)::geography
) < 2000;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on fasilitas (cost=0.00..7518.00 rows=200 width=23) (actual time=31.246..35.928 rows=39 loops=1)
2	Filter: (st_distance((geom)::geography, '0101000020E6100000D656EC2FBB2F5A402041F163CCDD07C0'::geography, true) < '2000'::double prec...
3	Rows Removed by Filter: 561
4	Planning Time: 13.915 ms
5	Execution Time: 36.781 ms

Kenapa lambat? Karena ST_Distance menghitung jarak satu per satu ke semua 600 baris dulu baru difilter. Akibatnya database tidak bisa pakai index dan terpaksa melakukan Seq Scan. Hasilnya waktu eksekusi sangat lama yaitu 36.781 ms.

Solusi Optimasi

Query diperbaiki dengan mengganti ST_Distance < 2000 menggunakan fungsi ST_DWithin:

```sql
SELECT nama, jenis
FROM fasilitas
WHERE ST_DWithin(
    geom::geography,
    ST_GeomFromText('POINT(104.7458 -2.9833)', 4326)::geography,
    2000
);
```

Query Query History

```sql
WHERE ST_DWithin(
    geom::geography,
    ST_GeomFromText('POINT(104.7458 -2.9833)', 4326)::geography,
    2000
);
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on fasilitas (cost=0.00..7516.50 rows=1 width=23) (actual time=13.182..16.653 rows=39 loops=1)
2	Filter: st_dwithin((geom)::geography, '0101000020E6100000D656EC2FBB2F5A402041F163CCDD07C0'::geography, '2000'::double precision,...
3	Rows Removed by Filter: 561
4	Planning Time: 0.157 ms
5	Execution Time: 17.044 ms

Kenapa lebih cepat? Fungsi ST_DWithin dirancang khusus untuk query radius dan secara internal menggunakan operator bounding box && sehingga bisa memanfaatkan index GiST. Hasilnya waktu turun menjadi 17.044 ms.

Speedup = (36.781 - 17.044) / 36.781 × 100 = 53.7%

9. Kesimpulan

Dari hasil praktikum ini, dapat disimpulkan beberapa hal penting:

1. **Spatial Index sangat penting untuk performa query spasial.** Tanpa index, database harus membaca semua baris satu per satu (Seq Scan). Dengan index GiST, database langsung menuju data yang relevan saja. Rata-rata speedup yang didapat pada praktikum ini adalah sekitar **34%**.
2. **Semakin besar data, semakin besar manfaat index.** Pada tabel dengan baris sedikit seperti kecamatan (8 baris), PostgreSQL kadang tetap pakai Seq Scan karena dianggap lebih efisien. Namun pada data yang besar, perbedaannya bisa ratusan kali lebih cepat.
3. **Cara penulisan query juga menentukan apakah index bisa digunakan.** Terbukti dari percobaan query lambat: meskipun index sudah ada, penggunaan ST_Distance membuat index tidak terpakai. Solusinya adalah menggunakan ST_DWithin yang

memang dirancang untuk memanfaatkan index.

4. **EXPLAIN ANALYZE adalah alat wajib untuk debugging query lambat.** Dengan melihat apakah query plan menunjukkan Seq Scan atau Index Scan, kita bisa langsung mengetahui apakah index kita benar-benar digunakan atau tidak.
5. **Best practice yang harus selalu diterapkan:**
   - Selalu buat GiST index pada kolom geometri
   - Gunakan ST_DWithin untuk query radius, bukan ST_Distance < x
   - Gunakan EXPLAIN ANALYZE setiap kali query terasa lambat.